

**LAPORAN PRAKTIKUM
STRUKTUR DATA**

**MODUL XIV
PENGENALAN GRAPH**



Disusun Oleh :

NAMA : Rikza Nur Zaki

NIM : 103112430030

Dosen

FAHRUDIN MUKTI WIBOWO

**PROGRAM STUDI STRUKTUR DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2025**

A. Teori Dasar GRAPH

Graph didefinisikan sebagai $G = (V, E)$, di mana V adalah himpunan vertex (simpul) dan E adalah himpunan edge (sisi penghubung). Graph tak berarah tidak memiliki arah pada edge, sedangkan graph berarah (directed) memiliki arah dari satu vertex ke vertex lain. Graph juga bisa memiliki bobot pada edge untuk kasus weighted graph. DFS (Depth-First Search) menelusuri sedalam mungkin sebelum mundur, diimplementasikan rekursif atau dengan stack. BFS (Breadth-First Search) menelusuri level demi level menggunakan queue, berguna untuk jalur terpendek pada unweighted graph. Kedua algoritma memerlukan visited array untuk hindari siklus.

B. Guided (berisi screenshot source code & output program disertai penjelasannya)

Guided 1

```
graf.h
#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>
using namespace std;

typedef char infoGraph;

struct ElmNode;
struct ElmEdge;

typedef ElmNode* adrNode;
typedef ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode node;
    adrEdge next;
};

struct Graph {
    adrNode first;
};

void CreateGraph(Graph &G);
adrNode AllocateNode(infoGraph X);
adrEdge AllocateEdge(adrNode N);

void InsertNode(Graph &G, infoGraph X);
adrNode FindNode(Graph G, infoGraph X);

void ConnectNode(Graph &G, infoGraph A, infoGraph B);

void PrintInfoGraph(Graph G);

void ResetVisited(Graph &G);
void PrintDFS(Graph &G, adrNode N);
void PrintBFS(Graph &G, adrNode N);

#endif
graf.cpp
```

```

✓ #include "graf.h"
✓ #include <queue>
✓ #include <stack>

✓ void CreateGraph(Graph &G) {
|   G.first = NULL;
| }

✓ adrNode AllocateNode(infoGraph X) {
|   adrNode P = new ElmNode;
|   P->info = X;
|   P->visited = 0;
|   P->firstEdge = NULL;
|   P->next = NULL;
|   return P;
| }

✓ adrEdge AllocateEdge(adrNode N) {
|   adrEdge P = new ElmEdge;
|   P->node = N;
|   P->next = NULL;
|   return P;
| }

✓ void InsertNode(Graph &G, infoGraph X) {
|   adrNode P = AllocateNode(X);
|   P->next = G.first;
|   G.first = P;
| }

✓ adrNode FindNode(Graph G, infoGraph X) {
|   adrNode P = G.first;
|   while (P != NULL) {
|       if (P->info == X)
|           return P;
|       P = P->next;
|   }
|   return NULL;
| }

✓ void ConnectNode(Graph &G, infoGraph A, infoGraph B) {
|   adrNode N1 = FindNode(G, A);
|   adrNode N2 = FindNode(G, B);

|   if (N1 == NULL || N2 == NULL) {
|       cout << "Node tidak ditemukan!\n";
|       return;
|   }

|   adrEdge E1 = AllocateEdge(N2);
|   E1->next = N1->firstEdge;
|   N1->firstEdge = E1;

|   adrEdge E2 = AllocateEdge(N1);
|   E2->next = N2->firstEdge;
|   N2->firstEdge = E2;
| }

```

```

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {
        cout << P->info << " -> ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << E->node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

```

```

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

```

```

void PrintDFS(Graph &G, adrNode N) {
    if (N == NULL)
        return;

    N->visited = 1;
    cout << N->info << " ";

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        if (E->node->visited == 0) {
            PrintDFS(G, E->node);
        }
        E = E->next;
    }
}

```

```

void PrintBFS(Graph &G, adrNode N) {
    if (N == NULL)
        return;

    queue<adrNode> Q;
    Q.push(N);

    while (!Q.empty()) {
        adrNode curr = Q.front();
        Q.pop();

        if (curr->visited == 0) {
            curr->visited = 1;
            cout << curr->info << " ";

            adrEdge E = curr->firstEdge;
            while (E != NULL) {
                if (E->node->visited == 0) {
                    Q.push(E->node);
                }
                E = E->next;
            }
        }
    }
}

```

main.cpp

```

#include "graf.h"
#include "graf.cpp"
#include <iostream>
using namespace std;

int main() {
    Graph G;
    CreateGraph(G);

    InsertNode(G, 'A');
    InsertNode(G, 'B');
    InsertNode(G, 'C');
    InsertNode(G, 'D');
    InsertNode(G, 'E');

    ConnectNode(G, 'A', 'B');
    ConnectNode(G, 'A', 'C');
    ConnectNode(G, 'B', 'D');
    ConnectNode(G, 'C', 'E');

    cout << "=== Struktur Graph ===\n";
    PrintInfoGraph(G);

    cout << "\n=== DFS dari Node A ===\n";
    ResetVisited(G);
    PrintDFS(G, FindNode(G, 'A'));

    cout << "\n\n=== BFS dari Node A ===\n";
    ResetVisited(G);
    PrintBFS(G, FindNode(G, 'A'));

    cout << endl;

    return 0;
}

```

Screenshots Output

```

=== Struktur Graph ===
E -> C
D -> B
C -> E A
B -> D A
A -> C B

=== DFS dari Node A ===
A C E B D

=== BFS dari Node A ===
A C B E D

```

Deskripsi:

Program diatas mengimplementasikan struktur data graph tak berarah, di mana setiap ElmNode menyimpan informasi node, pointer ke daftar sisi (firstEdge), dan pointer ke node berikutnya, sedangkan setiap sisi (ElmEdge) menyimpan pointer ke node tujuan. Fungsi CreateGraph menginisialisasi graph kosong, AllocateNode dan AllocateEdge digunakan untuk alokasi memori node dan edge, InsertNode menambahkan node baru ke dalam graph, dan FindNode mencari node berdasarkan label. Fungsi ConnectNode menghubungkan dua node secara dua arah dengan menambahkan edge pada masing-masing node sehingga membentuk graph tak berarah. PrintInfoGraph menampilkan struktur graph beserta hubungan antar node. Traversal graph diimplementasikan melalui PrintDFS dan PrintBFS, dengan ResetVisited untuk mengatur ulang status kunjungan sebelum traversal. Pada fungsi main menampilkan struktur graph serta hasil traversal

DFS dan BFS, dan hasil nya berada di code ss atas.

C. Unguided/Tugas (berisi screenshot source code & output program disertai penjelasannya)

Unguided 1

```
graph.h
#ifndef GRAPH_H_INCLUDED
#define GRAPH_H_INCLUDED

#include <iostream>

using namespace std;

typedef char infoGraph;
typedef struct ElmNode* adrNode;
typedef struct ElmEdge* adrEdge;

struct ElmNode {
    infoGraph info;
    int visited;
    adrEdge firstEdge;
    adrNode next;
};

struct ElmEdge {
    adrNode Node;
    adrEdge next;
};

struct Graph {
    adrNode first;
};

struct QueueNode {
    adrNode data;
    QueueNode* next;
};

struct Queue {
    QueueNode* head;
    QueueNode* tail;
};

void CreateGraph(Graph &G);
void InsertNode(Graph &G, infoGraph X);
adrNode findNode(Graph G, infoGraph X);
void ConnectNode(adrNode N1, adrNode N2);
void PrintInfoGraph(Graph G);
void ResetVisited(Graph &G);
void PrintDFS(Graph G, adrNode N);
void PrintBFS(Graph G, adrNode N);

void initQueue(Queue &Q);
void enqueue(Queue &Q, adrNode P);
adrNode dequeue(Queue &Q);
bool isEmpty(Queue Q);
#endif
```

graph.cpp

```

#include "graph.h"

void CreateGraph(Graph &G) {
    G.first = NULL;
}

void InsertNode(Graph &G, infoGraph X) {
    adrNode newNode = new ElmNode;
    newNode->info = X;
    newNode->visited = 0;
    newNode->firstEdge = NULL;
    newNode->next = NULL;

    if (G.first == NULL) {
        G.first = newNode;
    } else {
        adrNode temp = G.first;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

adrNode findNode(Graph G, infoGraph X) {
    adrNode P = G.first;
    while (P != NULL) {
        if (P->info == X) return P;
        P = P->next;
    }
    return NULL;
}

void ConnectNode(adrNode N1, adrNode N2) {
    if (N1 != NULL && N2 != NULL) {
        adrEdge E1 = new ElmEdge;
        E1->Node = N2;
        E1->next = N1->firstEdge;
        N1->firstEdge = E1;

        adrEdge E2 = new ElmEdge;
        E2->Node = N1;
        E2->next = N2->firstEdge;
        N2->firstEdge = E2;
    }
}

void PrintInfoGraph(Graph G) {
    adrNode P = G.first;
    while (P != NULL) {
        cout << "Node " << P->info << ": ";
        adrEdge E = P->firstEdge;
        while (E != NULL) {
            cout << "-> " << E->Node->info << " ";
            E = E->next;
        }
        cout << endl;
        P = P->next;
    }
}

```

```

void ResetVisited(Graph &G) {
    adrNode P = G.first;
    while (P != NULL) {
        P->visited = 0;
        P = P->next;
    }
}

void PrintDFS(Graph G, adrNode N) {
    if (N == NULL || N->visited == 1) return;

    cout << N->info << " ";
    N->visited = 1;

    adrEdge E = N->firstEdge;
    while (E != NULL) {
        PrintDFS(G, E->Node);
        E = E->next;
    }
}

void initQueue(Queue &Q) {
    Q.head = NULL;
    Q.tail = NULL;
}

void enqueue(Queue &Q, adrNode P) {
    QueueNode* newNode = new QueueNode;
    newNode->data = P;
    newNode->next = NULL;
    if (Q.head == NULL) {
        Q.head = newNode;
        Q.tail = newNode;
    } else {
        Q.tail->next = newNode;
        Q.tail = newNode;
    }
}

adrNode dequeue(Queue &Q) {
    if (Q.head == NULL) return NULL;
    QueueNode* temp = Q.head;
    adrNode res = temp->data;
    Q.head = Q.head->next;
    if (Q.head == NULL) Q.tail = NULL;
    delete temp;
    return res;
}

bool isEmpty(Queue Q) {
    return Q.head == NULL;
}

```



```

void PrintBFS(Graph G, adrNode N) {
    if (N == NULL) return;

    Queue Q;
    initQueue(Q);

    N->visited = 1;
    enqueue(Q, N);

    while (!isEmpty(Q)) {
        adrNode curr = dequeue(Q);
        cout << curr->info << " ";

        adrEdge E = curr->firstEdge;
        while (E != NULL) {
            if (E->Node->visited == 0) {
                E->Node->visited = 1;
                enqueue(Q, E->Node);
            }
            E = E->next;
        }
    }
}

```

main.cpp

```

#include "graph.h"
#include "graph.cpp"

int main() {
    Graph G;
    CreateGraph(G);

    char nodes[] = {'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'};
    for (int i = 0; i < 8; i++) {
        InsertNode(G, nodes[i]);
    }

    ConnectNode(findNode(G, 'A'), findNode(G, 'B'));
    ConnectNode(findNode(G, 'A'), findNode(G, 'C'));
    ConnectNode(findNode(G, 'B'), findNode(G, 'D'));
    ConnectNode(findNode(G, 'B'), findNode(G, 'E'));
    ConnectNode(findNode(G, 'C'), findNode(G, 'F'));
    ConnectNode(findNode(G, 'C'), findNode(G, 'G'));
    ConnectNode(findNode(G, 'D'), findNode(G, 'H'));
    ConnectNode(findNode(G, 'E'), findNode(G, 'H'));
    ConnectNode(findNode(G, 'F'), findNode(G, 'H'));
    ConnectNode(findNode(G, 'G'), findNode(G, 'H'));

    cout << "Struktur Graph:" << endl;
    PrintInfoGraph(G);

    cout << "\nHasil DFS (Mulai dari A):" << endl;
    ResetVisited(G);
    PrintDFS(G, findNode(G, 'A'));
    cout << endl;

    cout << "\nHasil BFS (Mulai dari A):" << endl;
    ResetVisited(G);
    PrintBFS(G, findNode(G, 'A'));
    cout << endl;

    return 0;
}

```

Screenshots Output

```

Struktur Graph:
Node A: -> C -> B
Struktur Graph:
Node A: -> C -> B
Node B: -> E -> D -> A
Node A: -> C -> B
Node B: -> E -> D -> A
Node B: -> E -> D -> A
Node C: -> G -> F -> A
Node C: -> G -> F -> A
Node D: -> H -> B
Node E: -> H -> B
Node F: -> H -> C
Node D: -> H -> B
Node E: -> H -> B
Node F: -> H -> C
Node G: -> H -> C
Node F: -> H -> C
Node G: -> H -> C
Node G: -> H -> C
Node H: -> G -> F -> E -> D

Hasil DFS (Mulai dari A):
A C G H F E B D

Hasil BFS (Mulai dari A):
A C B G F E D H

```

Deskripsi:

Program diatas mengimplementasikan struktur data graph tak berarah. program masih menggunakan struktur dan fungsi yang sama seperti fungsi PrintDFS, PrintBFS, serta fungsi-fungsi pendukung seperti ResetVisited, enqueue, dan dequeue untuk mengontrol alur penelusuran graf. Fungsi PrintDFS untuk memanggil dirinya sendiri pada setiap fungsi yang belum dikunjungi, sedangkan PrintBFS memanfaatkan struktur antrian untuk menyimpan simpul yang akan dikunjungi berikutnya. Pada fungsi main menampilkan struktur graph serta hasil traversal DFS dan BFS, dan hasil nya berada di code ss atas.

D. Kesimpulan

Berdasarkan laprak diatas, dapat disimpulkan bahwa Graph adalah struktur data yang terdiri dari himpunan vertex (simpul) dan edge (sisi) yang dapat diimplementasikan menggunakan Multi Linked List untuk merepresentasikan hubungan antara simpul. Graph tak berarah dapat dijalankan melalui operasi seperti penyisipan simpul, penyambungan sisi, dan penelusuran menggunakan algoritma Depth-First Search (DFS), serta Breadth-First Search (BFS) yang memanfaatkan struktur antrian (queue). Penggunaan penanda visited pada setiap simpul dalam algoritma penelusuran tersebut untuk menghindari terjadinya perulangan yang tak terbatas saat program dijalankan.

E. Referensi

<https://www.geeksforgeeks.org/cpp/cpp-program-to-implement-adjacency-list/>
<https://www.geeksforgeeks.org/cpp/implementation-of-graph-in-cpp/>